PA5: Spiral Cipher

CSE 3150 — Fall 2025

Overview

In this assignment, you will implement a simple character-based encryption system using a spiral cipher. The cipher maps each printable ASCII character (from " " to "∼") to another character in a fixed spiral pattern. You will implement functions to create the cipher map, encrypt and decrypt messages, compute character frequencies, and find common characters between two strings. All functions must be implemented in `PA5.cpp`.

Part 1: Create the Spiral Cipher Map

Implement the following function:

```
std::map<char, char> ECCreateSpiralCipherMap();
```

This cipher maps each printable ASCII character (from `" "` to `"∼"`) into a square matrix. After you create the matrix, traverse it in a clockwise spiral order, mapping each character to the one next to it in the spiral. The last character will map to the first character to maintain the pattern. This function should return a map of type `map<char, char>` representing the cipher. Your implementation should ensure that the cipher map contains exactly 95 entries, corresponding to all printable ASCII characters.

Part 2: Encrypt and Decrypt Messages

Implement the following functions:

```
std::string ECEncryptMessage(...);
std::string ECDecryptMessage(...);
```

In order to transform each character, use the cipher map. Ensure that the characters in the map are left unchanged. Decryption is implemented by reversing the mapping.

Part 3: Analyze Encrypted Messages

Implement the following functions:

```
std::map<char, int> ECCharacterFrequency(...);
std::vector<char> ECCommonCharacters(...);
```

- `ECCharacterFrequency`: returns a frequency map of characters in the message.
- `ECCommonCharacters`: returns characters that appear in both messages (no duplicates).

Function Specifications

1. `std::map<char, char> ECCreateSpiralCipherMap()`

Returns a mapping from each printable ASCII character to another character in a spiral pattern. The mapping wraps around such that "∼" maps to " ".

Example Output (partial):

```
(' ' → '!'), ('!' → '"') ... ('~' → ' ')
```

2. `std::string ECEncryptMessage(const std::string& msg, const std::map<char, char>& cipher)`

Encrypts a message by replacing each character using the cipher map. Characters not in the map (e.g., newline, tab) should remain unchanged.

Input: `"C++ is cool!"` **Output:** `"D,,jt!dppl¨"`

3. `std::string ECDecryptMessage(const std::string& encrypted, const std::map<char, char>& cipher)`

Decrypts a message by reversing the cipher map.

Input: `"D,,jt!dppl¨"` **Output:** `"C++ is cool!"`

4. `std::map<char, int> ECCharacterFrequency(const std::string& msg)`

Returns a frequency map of characters in the input string.

Input: `"C++ is cool!"` **Output:**

```
'C' → 1
'+' → 2
' ' → 2
'i' → 1
's' → 1
'c' → 1
'o' → 2
'l' → 1
'!' → 1
```

5. `std::vector<char> ECCommonCharacters(const std::string& a, const std::string& b)`

Returns a vector of characters that appear in both strings. Order does not matter.

Input: `"encrypted"`, `"decrypted"` **Output:** {'e', 'c', 'r', 'y', 'p', 't', 'd'}

Edge Cases

Edge cases include the following, Non-printable characters, such as (e.g., `\n`, `\t`) which should remain unchanged, empty strings (should return empty outputs), and ensuring the cipher map contains exactly 95 entries.

Compilation

Your code will be compiled with:

```
g++ -std=c++17 -Wall -Wextra -O2 -o test test-main.cpp PA5.cpp
```

Submission

Submit only `PA5.cpp`.